Object Detection with Python using YOLOv8

Project #29 – Algorithm and Dataset Specification

---

1. Introduction

The project **"Object Detection with Python using YOLOv8"** proposes the development of a real-time intelligent object detection system capable of processing **webcam streams, images, and video inputs** using the advanced **YOLOv8 deep learning model**. The main aim of this project is to design a smart system that can automatically detect, classify, and localize multiple objects in a visual scene with speed and accuracy.

Object detection plays a major role in many modern applications such as **surveillance systems, autonomous vehicles, traffic monitoring, industrial automation, smart retail, public safety, and intelligent monitoring systems**. Manual observation of objects in images and video streams is often time-consuming and error-prone. Therefore, this project is designed to provide an AI-based automated solution for detecting objects efficiently in real time.

The project is planned to be implemented using **Python**, **OpenCV**, and the **Ultralytics YOLOv8 framework**. The proposed system is expected to process input frames continuously, identify multiple objects present in the scene, and display the detection results using bounding boxes, class labels, and confidence scores.

---

2. Proposed Code Implementation

The following Python code represents the proposed basic implementation of the object detection system:

```
import cv2
from ultralytics import YOLO
import time

# Initialize YOLOv8n model
model = YOLO("yolov8n.pt")

# Webcam capture
cap = cv2.VideoCapture(0)
fps_start = time.time()

print("YOLOv8 Object Detection Active - Project #29")
print("Press 'q' to exit | 80 COCO Classes | conf=0.25, iou=0.45")
```

```
while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Real-time inference
    results = model(frame, conf=0.25, iou=0.45, verbose=False)

    # Render detections with bounding boxes + confidence
    annotated_frame = results[0].plot()

    # FPS counter
    fps = 1.0 / (time.time() - fps_start)
    fps_start = time.time()
    cv2.putText(annotated_frame, f"FPS: {fps:.1f}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 255), 2)

    cv2.imshow("YOLOv8 Object Detection - Project #29", annotated_frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

This code is intended to demonstrate the basic working of the object detection system using a **webcam input source**. The model detects objects frame by frame and displays the results in real time.

---

3. Algorithm Specification

A. Step-by-Step Procedure

1. **Start the system**.
2. Import the required Python libraries such as:
    - OpenCV
    - Ultralytics YOLOv8
    - Time module
3. Load the pre-trained **YOLOv8n model** (yolov8n.pt).
4. Initialize the input source using:

    - Webcam
    - Image file
    - Video file
5. Capture the frame from the selected input source.
6. Preprocess the frame if required for detection.
7. Pass the frame to the **YOLOv8 model**.
8. The model analyzes the input and detects all possible objects.
9. For each detected object, the system identifies:
    - Object class name
    - Bounding box coordinates
    - Confidence score
10. Apply **Non-Maximum Suppression (NMS)** to remove duplicate detections.
11. Draw bounding boxes around the detected objects.
12. Display object labels and confidence values on the output screen.
13. Calculate and display the frame processing speed (FPS).
14. Continue the same process for every frame in real time.
15. Stop the program when the user exits.
16. Release the webcam and close all windows.
17. **End the system**.

B. Logic or Method Used

The proposed project uses a **deep learning-based single-stage object detection approach**. The main logic of the system is to analyze the visual input and automatically identify and localize objects present in the frame.

Unlike traditional image processing methods, this system uses a **trained convolutional neural network** to perform:

- **Object classification**
- **Object localization**
- **Confidence scoring**

all in a **single forward pass**, which makes the detection process fast and efficient.

The object detection logic is based on:

- reading the input frame
- running inference using YOLOv8
- filtering predictions based on confidence threshold
- displaying valid detections visually

This approach makes the system suitable for **real-time applications**.

C. Model or Technique Applied

The main model proposed for this project is:

YOLOv8n (You Only Look Once – Version 8 Nano Model)

YOLOv8 is one of the latest and most efficient object detection models developed for real-time AI applications. It is designed to detect multiple objects in an image or video frame in a single pass, making it both fast and accurate.

Model Architecture

The YOLOv8n model consists of:

- **Backbone:** For feature extraction
- **Neck:** For feature fusion
- **Detection Head:** For predicting object classes and bounding boxes

Techniques Applied

The project uses the following techniques:

- Deep Learning
- Computer Vision
- Object Classification
- Object Localization
- Bounding Box Detection
- Confidence Thresholding
- Non-Maximum Suppression (NMS)

Advantages of YOLOv8

- Fast object detection
- Good accuracy

- Real-time performance
- Supports multiple object classes
- Easy integration with Python

The pre-trained model supports **80 common object classes** from the COCO dataset.

D. Process Flow of the System

The proposed system follows the below process flow:

Input Source → Frame Capture → Preprocessing → YOLOv8 Detection → Filtering → Bounding Box Rendering → Output Display

Flow Explanation

- The user provides input through webcam, image, or video.
- The system captures the frame.
- The frame is processed by the YOLOv8 model.
- The model detects and classifies the objects.
- Duplicate or low-confidence detections are filtered.
- The final result is displayed with labels and bounding boxes.

This flow allows the system to perform **continuous real-time object detection** efficiently.

---

4. Dataset Specification

A. Source of Data

The proposed project uses the **COCO (Common Objects in Context) 2017 Dataset** indirectly through the **pre-trained YOLOv8 model**.

Dataset Source

- **Dataset Name:** COCO 2017 Dataset
- **Model Source:** Ultralytics YOLOv8
- **Official Dataset Source:** COCO Dataset

The COCO dataset is widely used in computer vision and contains a large number of real-world images with multiple object categories.

B. Number of Records / Samples

The COCO 2017 dataset approximately contains:

- **Training Images:** 118,000+
- **Validation Images:** 5,000+
- **Test Images:** 20,000+ (approx.)
- **Object Annotations:** 800,000+ instances
- **Number of Classes:** 80 object classes

This large dataset helps the model detect objects under different lighting, backgrounds, and real-world conditions.

C. Features or Attributes (Columns)

The dataset is image-based and contains annotation details for each object.

Main attributes include:

- Image ID
- File Name
- Category ID
- Object Class Name
- Bounding Box Coordinates:
    - x-coordinate
    - y-coordinate
    - width
    - height
- Area
- Object label information

Examples of object classes

- Person
- Car
- Dog
- Cat
- Chair
- Laptop

- Bottle
- Bus
- Bicycle
- Mobile phone

The YOLOv8 model can detect **80 common object categories** from the COCO dataset.

D. Training and Testing Data Split

Since the proposed project uses a **pre-trained YOLOv8 model**, the training has already been completed by the model developers.

General dataset split

- **Training Set:** 118,000+ images
- **Validation Set:** 5,000+ images
- **Test Set:** 20,000+ images (benchmark/testing)

Project Usage

For this project, the model will mainly be used for:

- Webcam object detection
- Image object detection
- Video object detection

So the project focuses on **implementation and inference**, rather than training the model from scratch.

E. Data Format or Type

The system is planned to support the following input formats:

Image Formats

- .jpg
- .jpeg
- .png

Video Formats

- .mp4
- .avi

- .mov

Live Input

- Webcam stream

Annotation Formats

- .json (COCO format)
- .txt (YOLO format)

The input data is primarily visual data used for object detection and classification tasks.

---

5. Expected / Reference Performance Metrics

The following are expected or reference-level performance metrics based on the YOLOv8n model:

- **Fast real-time inference**
- **Support for 80 COCO object classes**
- **Lightweight model suitable for system implementation**
- **Detection on webcam, images, and videos**
- **Bounding box generation with confidence scores**
- **Efficient object recognition under normal conditions**

These values may vary depending on:

- system hardware
- camera quality
- lighting conditions
- input resolution

---

6. Conclusion

The project **"Object Detection with Python using YOLOv8"** is proposed as a real-time intelligent object detection system that uses deep learning and computer vision to identify multiple objects from visual input. The algorithm is designed to process frames efficiently and detect objects accurately using the YOLOv8 model.

This project provides practical exposure to **Artificial Intelligence, Deep Learning, Computer Vision, and Python programming**, and can be extended in the future for applications such as:

- Smart surveillance
- Vehicle detection
- People counting
- Attendance monitoring
- Security systems
- Industrial automation

Thus, the project serves as an effective academic implementation of modern AI-based object detection technology.